Unmixing the Music

The Sound of Sparsity

Sparse, Low-Rank, and Deep: A Compressed-Sensing Approach to Source Separation

Taka Khoo • Thayer School of Engineering, Dartmouth College • ENGS 109

DARTMOUTH ENGINEERING

The Problem in One Sentence

Given only a finished song, recover its hidden tracks, the **vocals** and the **accompaniment**, that were summed together when it was mixed.

This is hard because mixing destroys information: countless track pairs add up to the exact same song. We need a **prior** that says which split is plausible.

Intuition: A Song Is a Picture of Sound

- **Spectrogram:** the short-time Fourier transform turns audio into an image: time on the x-axis, pitch on the y-axis, brightness = energy.
- **Vocals are sparse:** a singing voice lights up only a few horizontal harmonic lines per instant, most of the image is dark.
- **Accompaniment is low-rank:** repeating chords and drums make the background a sum of a few recurring patterns.

Sparse and **low-rank** are exactly the structures that **compressed sensing** knows how to recover, so the same math that reconstructs MRI and photographs from few measurements applies to audio.

The Structure Each Method Exploits

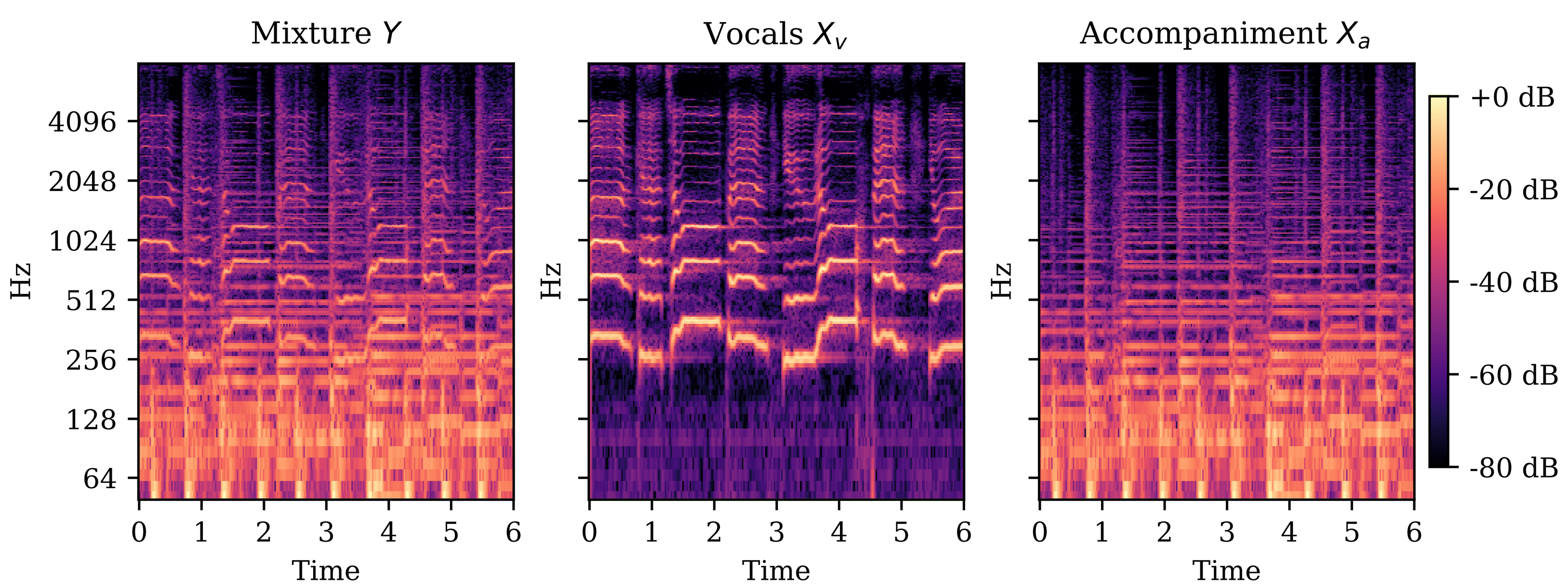


Figure 1: Mixture (left), the vocal track (center, sparse harmonic scaffold), and the accompaniment (right, denser and low-rank). Our methods reverse the sum.

When Is Recovery Guaranteed?

For $\boldsymbol{y} = \boldsymbol{A}\boldsymbol{x}$ with $\boldsymbol{x}$ s-sparse, the convex program $\min\|\boldsymbol{z}\|_1$ s.t. $\boldsymbol{A}\boldsymbol{z} = \boldsymbol{y}$ returns $\boldsymbol{x}$ exactly under *any* of three conditions:

$$\underbrace{\operatorname{spark}(\boldsymbol{A}) > 2s}_{\text{uniqueness}}, \quad \underbrace{\delta_{2s} < \sqrt{2}-1}_{\text{RIP}}, \quad \underbrace{s < \tfrac{1}{2}\left(1+\tfrac{1}{\mu(\boldsymbol{A})}\right)}_{\text{coherence}}.$$

- **RIP:** $(1-\delta_s)\|\boldsymbol{x}\|_2^2 \le \|\boldsymbol{A}\boldsymbol{x}\|_2^2 \le (1+\delta_s)\|\boldsymbol{x}\|_2^2$.
- **Coherence:** $\mu(\boldsymbol{A}) = \max_{i\neq j}|\langle \boldsymbol{a}_i, \boldsymbol{a}_j\rangle|$; random $\boldsymbol{A}$ needs only $m \ge Cs\log(N/s)$ rows.
- **Low-rank:** nuclear norm $\|\boldsymbol{X}\|_* = \sum_i \sigma_i$ is the convex surrogate for rank.

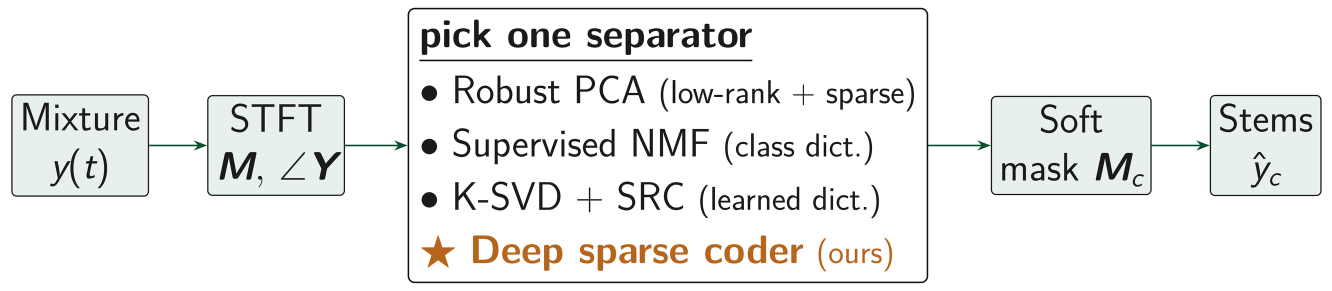
Why Music Lands in the Recoverable Regime

The numbers from a real clip (f_s=22 kHz, 2048-pt STFT) put us inside the guarantees above:

- **Dimensions:** F=1025 frequency bins, $T\approx$340 frames per 8 s, so the spectrogram is a 1025×340 image.
- **Vocals are very sparse:** a frame activates $s\approx$10 harmonic atoms out of a K=96 dictionary, so the active fraction is ∼10%.
- **Accompaniment is very low rank:** the magnitude spectrum's singular values collapse after $r\approx$4–8 components (Fig. 5, right), $r \ll \min(F, T)$.

Sparse vocals + low-rank accompaniment is exactly the low-rank-plus-sparse model that principal component pursuit provably separates.

One Front End, Four Separators



Each separator outputs a **time-frequency mask** $\boldsymbol{M}_c$ that keeps a fraction of every spectrogram cell; the original **mixture phase** is reused to turn the masked magnitude back into audio.

Classical Recovery Stages

Robust PCA. Split the spectrogram into low-rank + sparse:

$$\min_{\boldsymbol{X},\boldsymbol{E}} \|\boldsymbol{X}\|_* + \lambda\|\boldsymbol{E}\|_1 \text{ s.t. } \boldsymbol{X}+\boldsymbol{E}=\boldsymbol{M}, \ \lambda = \frac{1}{\sqrt{\max(F,T)}}.$$

Low-rank $\boldsymbol{X}$ = accompaniment, sparse $\boldsymbol{E}$ = voice. No training.

Supervised NMF. Learn a non-negative dictionary per source; KL multiplicative updates $\boldsymbol{H} \leftarrow \boldsymbol{H} \odot \frac{\boldsymbol{D}^\top(\boldsymbol{M} \oslash \boldsymbol{DH})}{\boldsymbol{D}^\top \boldsymbol{1}}$, fix $\boldsymbol{D} = [\boldsymbol{D}_v | \boldsymbol{D}_a]$ at test time.

K-SVD + SRC. Overcomplete dictionaries via rank-one atom updates; route each frame by which source reconstructs it best, $c^\star = \arg\min_c \|\boldsymbol{y} - \boldsymbol{D}_c\boldsymbol{\alpha}_c\|_2$.

Deep Sparse Coder (the New Method)

Idea. A deep network *is* a sparse-recovery solver with its steps unrolled into layers. We learn the dictionaries by backprop. Each module stacks a *fat* (upsampling) then *tall* (downsampling) non-negative code:

$$\boldsymbol{\alpha}^{(1)} = \arg\min_{\boldsymbol{\alpha}\ge 0} \tfrac{1}{2}\|\boldsymbol{x} - \boldsymbol{D}^{(1)}\boldsymbol{\alpha}\|_2^2 + \lambda_1\|\boldsymbol{\alpha}\|_1 + \tfrac{\lambda_2}{2}\|\boldsymbol{\alpha}\|_2^2,$$

solved by K unrolled proximal-gradient steps (all matmul + clamp, runs on GPU):

$$\boldsymbol{\alpha}_k = \left[\boldsymbol{z}_k - \tfrac{1}{L}\boldsymbol{D}^\top(\boldsymbol{D}\boldsymbol{z}_k - \boldsymbol{x}) - \tfrac{\lambda_1}{L}\right]_+.$$

Training. Stack 4 modules + sigmoid; minimize $\|(\boldsymbol{M}_v - \boldsymbol{M}_v^{\mathrm{IRM}}) \odot \boldsymbol{M}\|_1 + \beta\sum_\ell \|\boldsymbol{\alpha}^{(\ell)}\|_1$. This is LISTA, specialized to non-negative codes and a masking output.

Inside the Deep Coder: One Composite Module

A **fat dictionary** lifts the frame into a *tall*, over-complete sparse code $\boldsymbol{\alpha}^{(1)}$; a **tall dictionary** then compresses it to a *short* code $\boldsymbol{\alpha}^{(2)}$ (a clustering effect). Four such modules and a sigmoid head emit the mask.

Why Unrolling Wins

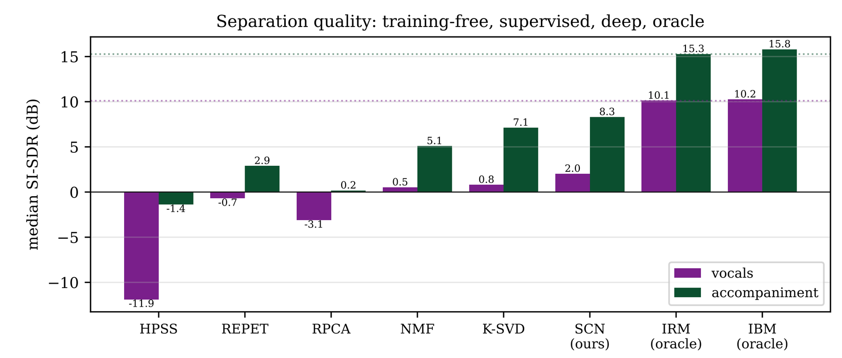
- **Every layer is a real solver step:** the equation above is one accelerated proximal-gradient (FISTA) iteration; run forever it hits the optimal $O(1/k^2)$ rate, but we stop at K learned steps.
- **Interpretable parameters:** the weights *are* dictionaries and shrinkage thresholds, not opaque filters.
- **Cheap and small:** one forward pass, 1.4M parameters, no test-time optimization, runs on a laptop GPU.
- **Temporal context:** a ±2-frame window feeds each prediction, so the coder sees local time structure.

Stereo as Joint Sparsity

Two channels, one support. Left and right share the same active sources, so their codes share *rows*. Simultaneous OMP scores atoms by the cross-channel row norm $\|(\boldsymbol{D}^\top\boldsymbol{R})_{n,:}\|_2$, and the shared support *improves* identifiability: $s < \frac{1}{2}(\operatorname{spark}(\boldsymbol{A}) - 1 + \operatorname{rank}(\boldsymbol{Y}))$.

Results on 25 MUSDB18 Test Tracks

Method	SI-SDR (dB) Vocals	SI-SDR (dB) Accomp.	Time (s)
HPSS (median)	−11.9	−1.4	0.25
REPET-SIM	−0.7	2.9	0.08
Robust PCA	−3.1	0.2	1.14
Supervised NMF	0.5	5.1	0.46
K-SVD + SRC	0.8	7.1	0.28
Deep coder (ours)	**2.0**	**8.3**	**0.12**
Oracle IRM / IBM	10.2	15.8	—



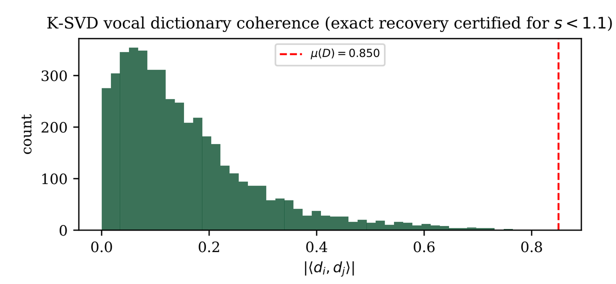
- **Structure-learning helps monotonically:** training-free (<0 dB) → learned dictionaries (+5 to +7 dB) → deep coder (+8.3 dB).
- **The deep coder wins on every axis:** best vocals (2.0 dB, 2.5× the best classical method), best accompaniment (8.3 dB), and the fastest learned model (0.12 s).

What the Knobs Do (Ablations)

NMF atoms K	16	32	64	128
Vocal SI-SDR	**2.2**	1.9	1.8	1.5
K-SVD s	3	5	10	15
Vocal SI-SDR	1.4	2.3	**2.7**	2.7
RPCA λ scale	0.5	1.0	1.5	2.0
Vocal SI-SDR	−3.9	−3.6	−3.2	**−2.3**

Smaller dictionaries generalize better (fewer atoms = stronger prior); **richer codes help** up to $s\approx$10, then saturate.

Honest Negative Result: Coherence



Learned dictionaries are **coherent** ($\mu \approx 0.85$), so the worst-case theorem certifies recovery only for $s \lesssim 1$, yet s=10 works in practice. **Average-case ≠ worst-case** for trained dictionaries.

Takeaways

- **Four methods, one idea:** all recover a *structured* spectrogram, so they compare fairly on one pipeline rather than as rival systems.
- **Learning more structure always helps:** training-free split < learned templates < a network whose layers *are* sparse-recovery steps.
- **The deep coder is the practical winner:** cleanest vocals and accompaniment, yet small (1.4M params) and quickest to run.
- **Theory guides, it does not guarantee:** the trained dictionaries break the worst-case conditions yet still work in practice.